Keywords: model compression • binary quantization • large language models • spherical coding

BiSCo-LLM

Lookup-Free Binary Spherical Coding Brings Sub-1-Bit LLM Compression Within Reach

By Yuantian Shao, Peisong Wang, Zhilei Liu, Chuangyi Li, Yuanteng Chen, Pengcheng Xie, Yiwu Yao, Zhihui Wei & Jian Cheng (Chinese Academy of Sciences; University of CAS; Huawei Technologies)

arXiv:2607.08643 • arXiv preprint, July 2026

Large language models are memory-bound: frontier models require tens of gigabytes of weight storage, creating an insurmountable barrier for on-device and edge deployment. Existing extreme-quantization methods face a fundamental trade-off—codebook-based approaches achieve good quality but require storing a codebook and paying a lookup cost at inference; binary approaches eliminate lookup overhead but suffer severe quality degradation. BiSCo-LLM resolves this trade-off by introducing Binary Spherical Coding (BSCo), a lookup-table-free scheme that matches 2-bit codebook quality at 1-bit storage by representing weights as binary combinations of spherical frame vectors.

AT A GLANCE

Problem: Extreme LLM quantization forces a choice between codebook quality (with lookup overhead) or binary speed (with poor accuracy).

Why it matters: On-device deployment of large models is blocked by memory bandwidth; sub-1-bit compression at competitive quality would unlock a new class of edge applications.

Method: Binary spherical coding maps weight blocks onto a pre-defined tight frame of ±1 vectors; decoded via XNOR-popcount, requiring no lookup table.

Result: LLaMA-3-8B at 1-bit storage within 1.6 perplexity points of 2-bit AQLM; 12× memory reduction versus FP16.

Evidence: Benchmarked on WikiText-2 perplexity and MMLU accuracy for LLaMA-3 (8B, 13B, 70B) and Mistral-7B.

The Big Picture

The deployment gap for large language models is primarily a memory problem. A 70-billion-parameter model in FP16 requires ∼140 GB of GPU memory — prohibitive for all but the largest datacenter installations and completely out of reach for edge devices. Quantization compresses weight values to fewer bits, reducing memory proportionally. At 4-bit (GPTQ, AWQ) and 2-bit (AQLM, QuIP) precision, existing methods achieve competitive accuracy, but even 2-bit precision requires ∼17 GB for a 70B model and a runtime codebook for reconstruction. True 1-bit methods (BitNet, OneBit) eliminate the codebook but degrade quality sharply because ±1 scalar representations cannot capture the heterogeneous magnitude structure of LLM weights.

BiSCo-LLM breaks out of this binary by separating the *structure* of the code (binary on a sphere) from its *expressive power* (controlled by the number of frame vectors K).

Why Existing Approaches Fall Short

Codebook methods (AQLM, QuIP, VQ-LLM): Represent weight blocks as indices into a learnable codebook. Quality is excellent — codebooks can approximate diverse weight distributions — but the codebook itself must be stored in memory (typically 65,536 entries × 32 floats = 2 MB per layer) and queried at every matrix-vector multiply via a memory-bandwidth-intensive lookup.

Binary methods (BitNet, OneBit): Represent weights as single ±1 scalars (possibly scaled per row or column). Zero lookup overhead and excellent throughput, but accuracy degrades sharply because all weight columns of different norms are mapped to the same code alphabet. The fundamental mismatch is that ±1 cannot represent graded magnitudes.

Vector quantization without lookup: No prior work achieves this combination. The key missing insight is that binary *combinations* of vectors, rather than individual binary scalars, can approximate rich high-dimensional distributions while still decoding via purely binary arithmetic.

Core Mechanism

A binary spherical code represents each weight column $w \in \mathbb{R}^d$ as a sparse binary combination of pre-defined frame vectors $\{b_k\}_{k=1}^K \subset \{+1, -1\}^d$ that form a tight frame:

$$\hat{w} = \frac{1}{\sqrt{K}} \sum_{k=1}^{K} z_k b_k, \quad z_k \in \{0, 1\}$$

The frame property $\sum_k b_k b_k^\top = (K/d) I_d$ guarantees that the reconstruction operator is an isometry in expectation, distributing reconstruction error uniformly across all weight directions regardless of the weight norm distribution.

At runtime, decoding $\hat{w}$ requires only XNOR-popcount operations between z and the frame matrix — no lookup table, no floating-point multiply at decode time.

Technical Formulation

The binary tight frame satisfies:

$$\min_{\{b_k\}\subset\{+1,-1\}^d} \left\| \sum_{k=1}^{K} b_k b_k^\top - \frac{K}{d} I_d \right\|_F^2$$

This Grassmannian binary packing problem is solved once offline via a greedy sequential algorithm. Given the fixed frame, per-weight compression minimizes reconstruction error:

$$z^* = \operatorname*{argmin}_{z\in\{0,1\}^K} \left\| w - \frac{1}{\sqrt{K}} \sum_{k=1}^{K} z_k b_k \right\|^2$$

During training with straight-through estimators:

$$\frac{\partial \mathcal{L}}{\partial z_k} \approx \frac{\partial \mathcal{L}}{\partial \hat{w}} \cdot \frac{b_k}{\sqrt{K}} \cdot \mathbf{1}[|z_k - 0.5| < 0.5]$$

The effective bit-rate is $R = (\log_2 K)/d$ bits per weight, tunable from ≈ 0.5 to ≈ 1.5 bits by varying $K \in [2^{d/3}, 2^{2d}]$.

Learning or Inference Procedure

1. **Frame construction (once):** Build a binary tight frame $\{b_k\}_{k=1}^K$ offline via greedy orthogonal pursuit.
2. **Initialization:** For each weight block W, solve the binary coding problem via fast iterative bit-flip optimization.
3. **Fine-tuning:** Train the binary codes z (not the frame) using STE gradients with a small calibration dataset (∼512 sequences).
4. **Inference:** Replace each weight block with its binary code. Matrix-vector products computed as $\hat{W}x \approx \frac{1}{\sqrt{K}} B_z x$ where $B_z x$ is computed via XNOR-popcount on binary hardware.

What the Guarantee Says

For a d-dimensional weight vector and a random binary tight frame with K vectors, the expected reconstruction error is:

$$\mathbb{E}\left[\|w - \hat{w}\|^2\right] = \|w\|^2 \cdot \left(1 - \frac{K}{d \cdot 2^K} \sum_{s=0}^{K} \binom{K}{s} \left|\frac{s}{K} - 0.5\right|\right)$$

This bound is tight and shows that reconstruction error vanishes as $K \to \infty$ (more frame vectors = higher quality), independently of the weight norm distribution.

Experimental Findings

Method (LLaMA-3-8B)	Bits	PPL
FP16	16	6.14
GPTQ-4bit	4	6.58
AQLM-2bit	2	7.21
BitNet-1bit	1	9.87
BiSCo-LLM 1bit	1	7.73
BiSCo-LLM 0.5bit	0.5	9.14

WikiText-2 perplexity (lower is better). BiSCo-LLM at 1-bit matches AQLM's 2-bit quality within 0.5 PPL at half the memory footprint, and outperforms BitNet by 2.1 PPL at the same storage budget.

Ablations and Interpretation

Frame size K: Quality improves monotonically with K up to $K \approx 2d$, after which reconstruction error saturates. The sweet spot for 1-bit effective compression is $K = d$: each weight column uses d binary bits to select d frame vectors, yielding exactly 1 bit/weight.

Attention vs. MLP layers: Attention weights are more compressible (quality gap vs. FP16 smaller by 15%) than MLP weights, consistent with prior quantization literature. BiSCo-LLM supports layer-specific K tuning.

XNOR-popcount throughput: On a Snapdragon X Elite mobile SoC, BiSCo-LLM achieves 3.4× higher token generation throughput versus INT4 GPTQ, demonstrating that the lookup-free design translates directly to real hardware efficiency.

Model scale: Perplexity gap vs. FP16 decreases at larger model sizes (70B model: 1.8 PPL gap vs. 1.6 PPL gap for 8B at 1-bit), suggesting BiSCo-LLM scales favorably.

Reference

Yuantian Shao, Peisong Wang, Zhilei Liu, Chuangyi Li, Yuanteng Chen, Pengcheng Xie, Yiwu Yao, Zhihui Wei, Jian Cheng. **BiSCo-LLM: Lookup-Free Binary Spherical Coding for Extreme Low-Bit Large Language Model Compression**. arXiv:2607.08643, July 2026. https://arxiv.org/abs/2607.08643